C Programming Language Functions

Computer Programming 1 – Semi-final Review Notes
Salazar Colleges of Science and Institute of Technology

1. What is a Function?

A **function** is a self-contained block of code that performs a specific task. It is the basic building block of modular programming in C.

Analogy: Think of a function like a recipe. You give it ingredients (inputs), it does the cooking (processing), and gives you the finished dish (output).

2. Why Do We Use Functions?

- **Modularity** – Break a big problem into smaller, manageable pieces.
- **Reusability** – Write the code once and use it many times.
- **Readability** – Programs become easier to understand and maintain.
- **Avoid code duplication** – No need to copy-paste the same code repeatedly.
- **Easier debugging** – You can test and fix one function at a time.

3. Types of Functions in C

1. Library (Built-in) Functions – Already available in C standard libraries.
Examples: `printf()`, `scanf()`, `sqrt()`, `strlen()`, `pow()`

2. User-defined Functions – Functions that **you** create.

4. Basic Structure of a User-defined Function

```
return_type function_name(parameter_list)
{
    // body of the function
    // statements
    return value;   // optional if return_type is void
}
```

5. Three Important Parts of a Function

Part	Purpose	Example
Function Prototype / Declaration	Tells the compiler about the function before it is used	`int add(int a, int b);`
Function Definition	The actual code of the function	See examples in later sections
Function Call	How you use/execute the function	`result = add(5, 3);`

6. Important Concepts

A. Parameters vs Arguments

- **Parameters** → Variables in the function definition (e.g., x and y)
- **Arguments** → Actual values passed when calling the function (e.g., `num1` and `num2`)

B. Return Type

- `int`, `float`, `char`, `double`, etc. → function returns a value
- `void` → function does **not** return any value

C. Pass by Value

C always uses **pass by value**. A copy of the argument is passed to the function. Changes inside the function do **not** affect the original variable (unless you use pointers – advanced topic).

7. Categories of Functions (Based on Parameters & Return)

1. No parameters, no return value → `void display();`
2. No parameters, with return value → `int getNumber();`
3. With parameters, no return value → `void printSum(int a, int b);`
4. With parameters and return value → `int multiply(int a, int b);`

8. Common Mistakes Students Make

- Forgetting the function prototype
- Mismatch between parameter types in prototype and definition
- Forgetting the `return` statement when the function has a return type
- Calling the function incorrectly (wrong number or type of arguments)
- Declaring the function after `main()` without a prototype

9. Best Practices

- Always write a clear function prototype at the top
- Use meaningful function names (`calculateAverage`, `displayMenu`, etc.)
- Keep functions short and focused on one task
- Comment your functions

Sample Programs

With Return Values and Without Return Values (void)

Sample Program 1: Function WITH Return Value

This function takes two integers, adds them, and **returns** the result to the calling function.

```
#include <stdio.h>

// Function Prototype
int add(int x, int y);

int main()
{
    int num1 = 10, num2 = 20, sum;

    // Function Call
    sum = add(num1, num2);

    printf("Sum = %d\n", sum);
    return 0;
}

// Function Definition
int add(int x, int y)
{
    int result = x + y;
    return result;   // returns the value to main()
}
```

Output: *Sum = 30*

Sample Program 2: Function WITH Return Value (Product)

This function multiplies two numbers and returns the product.

```
#include <stdio.h>

// Function Prototype
int multiply(int a, int b);

int main()
{
    int x = 5, y = 7, product;

    product = multiply(x, y);
    printf("Product = %d\n", product);
    return 0;
}

// Function Definition
int multiply(int a, int b)
{
    return a * b;   // returns the product
}
```

Output: *Product = 35*

Sample Program 3: Function WITHOUT Return Value (void) – No Parameters

This function simply displays a message. It does not take parameters and does not return any value.

```
#include <stdio.h>

// Function Prototype
void displayMessage();

int main()
{
    displayMessage();   // Function Call
    return 0;
}

// Function Definition
void displayMessage()
{
    printf("Welcome to Computer Programming 1!\n");
    printf("Salazar Colleges of Science and Institute of Technology\n");
    // No return statement needed for void functions
}
```

Output:
Welcome to Computer Programming 1!
Salazar Colleges of Science and Institute of Technology

Sample Program 4: Function WITHOUT Return Value (void) – With Parameters

This function receives two numbers, computes the sum inside, and prints it. It does **not** return any value to main().

```
#include <stdio.h>

// Function Prototype
void printSum(int a, int b);

int main()
{
    int num1 = 15, num2 = 25;

    printSum(num1, num2);   // Function Call
    return 0;
}

// Function Definition
void printSum(int a, int b)
{
    int sum = a + b;
    printf("The sum of %d and %d is %d\n", a, b, sum);
    // No return statement – this is a void function
}
```

Output: *The sum of 15 and 25 is 40*

Sample Program 5: Combining Both Types

This program shows both a function that returns a value and a void function that only displays information.

```
#include <stdio.h>

// Function Prototypes
float calculateAverage(float a, float b, float c);
void displayResult(float avg);

int main()
{
    float n1 = 85.5, n2 = 90.0, n3 = 78.5;
    float average;

    average = calculateAverage(n1, n2, n3);  // returns a value
    displayResult(average);                   // void function

    return 0;
}

// Function WITH return value
float calculateAverage(float a, float b, float c)
{
    float avg = (a + b + c) / 3.0;
    return avg;
}

// Function WITHOUT return value (void)
void displayResult(float avg)
{
    printf("The average grade is: %.2f\n", avg);

    if (avg >= 75.0)
        printf("Status: PASSED\n");
    else
        printf("Status: FAILED\n");
}
```

Output:
The average grade is: 84.67
Status: PASSED

Quick Review / Possible Oral Questions

1. What is the difference between a function prototype and a function definition?
2. Explain pass by value with an example.
3. When do we use `void` as a return type?
4. What will happen if you forget to write a function prototype?
5. Give an example of a function that takes two integers and returns their product.
6. Differentiate between library functions and user-defined functions.
7. What is the difference between parameters and arguments?